

Java Stream API — Array Operations Notes

Topic: IntStream ka use karke array par common operations | Hinglish notes + practice problems

1. Stream API Kya Hai?

Java 8 me **Stream API** introduce hui thi jo collections/arrays par functional style me operations karne deti hai — bina manual loop likhe. Stream khud data store nahi karta, ye sirf ek pipeline hai jisme data flow hota hai aur us par operations (filter, map, sort, etc.) apply hote hain.

Primitive int array ke liye hum **Arrays.stream(arr)** use karte hain, jo ek **IntStream** return karta hai. IntStream specially designed hai int values ke liye (auto-boxing ka overhead avoid karta hai, aur sum(), average(), max(), min() jaise built-in methods deta hai).

Stream operations do type ke hote hain:

- **Intermediate operations** — filter(), map(), sorted(), distinct(). Ye ek naya stream return karte hain, isliye chain kar sakte ho (lazy — tabhi execute hote hain jab terminal operation call ho).
- **Terminal operations** — sum(), max(), min(), count(), toArray(), collect(). Ye stream ko "consume" karke final result dekar hain. Ek terminal operation ke baad stream dobara use nahi ho sakta.

2. Important Methods — Ek-Ek Karke Samjho

Arrays.stream(arr)

Int array ko IntStream me convert karta hai. Isi se pura pipeline shuru hota hai.

.sorted()

Stream ke elements ko ascending order me sort karta hai (natural ordering). Original array change nahi hota — naya sorted stream milta hai.

.toArray()

Stream ko wapas int[] array me convert kar deta hai. Ye terminal operation hai.

.sum()

Saare elements ka total sum return karta hai (int).

.max() / .min()

Stream ka sabse bada / sabse chhota element return karta hai, lekin type **OptionalInt** me — isliye .getAsInt() lagana padta hai value nikalne ke liye. Optional isliye kyunki agar stream empty ho to max/min exist hi nahi karega.

.getAsInt()

OptionalInt se actual int value nikalta hai. Agar stream empty tha to ye NoSuchElementException throw karega — isliye real code me pehle isPresent() check karna best practice hai.

.count()

Stream me kitne elements hain, wo long value me return karta hai (isliye humein (int) cast karna pada average nikalne ke liye).

.map(x -> x*x)

Har element par ek function apply karta hai aur transform karke naya stream deta hai — yaha har number ka square nikala gaya.

.filter(x -> x%2==0)

Sirf wahi elements pass hone deta hai jo condition (predicate) satisfy karte hain — yaha sirf even numbers.

.distinct()

Duplicate elements ko hata deta hai, sirf unique values rakhta hai.

Chaining (filter → map → distinct → sum)

Multiple intermediate operations ko ek ke baad ek jodh sakte ho — pipeline banti hai jo order me execute hoti hai, aur end me ek terminal operation lagake result nikalte ho.

3. Diya Gaya Code — Line by Line Samjho

Neeche wahi code hai jo aapne likha tha (thoda clean karke), har block ke baad Hinglish me explanation di gayi hai ki wo line kya kar rahi hai.

```
int[] arr = {1,2,3,4,5,6,6393,5,3,6,2,5,6,3,5,6,4,6,6};
int[] sort = Arrays.stream(arr).sorted().toArray();
System.out.println(Arrays.toString(sort));
```

Array ko stream me convert karke sorted() se ascending order me sort kiya, fir toArray() se wapas normal array bana liya taaki print kar saken.

```
int sum = Arrays.stream(arr).sum();
System.out.println(sum);

int max = Arrays.stream(arr).max().getAsInt();
System.out.println(max);

int min = Arrays.stream(arr).min().getAsInt();
System.out.println(min);
```

Sabse simple terminal operations — sum(), max().getAsInt(), min().getAsInt(). Note: har baar Arrays.stream(arr) fresh call kiya gaya hai kyunki ek stream ek hi baar consume ho sakta hai.

```
long elements = Arrays.stream(arr).count();
System.out.println((int)(sum / elements));
```

count() array ki length jaisa hi result deta hai (yaha use karne ki jagah simply arr.length bhi use kar sakte the). sum/elements se average nikala — isme int division ho rahi hai isliye decimal part cut jayega (truncation), round nahi hoga.

```
int[] sq = Arrays.stream(arr).map(x -> x*x).toArray();
System.out.println(Arrays.toString(sq));
```

map() se har element ka square nikala gaya — pura array transform ho gaya.

```
int[] even = Arrays.stream(arr).filter(x -> x%2==0).distinct().toArray();
System.out.println(Arrays.toString(even));

int[] pos = Arrays.stream(arr).filter(x -> x%2==0).map(x -> x*x).distinct().toArray();
System.out.println(Arrays.toString(pos));

int sumOfSq = Arrays.stream(arr).filter(x -> x%2==0).map(x -> x*x).distinct().sum();
System.out.println(sumOfSq);
```

Yaha teeno operations chain ho rahe hain: pehle filter se even numbers, fir distinct se duplicates hataye, phir map se square kiya. Teesri line me unhi unique even squares ka sum bhi nikal liya sum() laga ke.

```
int maxi = Arrays.stream(arr).filter(x -> x%2==0).max().getAsInt();
System.out.println(maxi);    // Largest Even No.

int smallPositive = Arrays.stream(arr).filter(x -> x>0).min().getAsInt();
System.out.println(smallPositive);

int[] sorted = Arrays.stream(arr).sorted().distinct().toArray();
System.out.println(Arrays.toString(sorted));
```

filter() ke baad max()/min() laga ke condition wale group me se hi largest/smallest nikala gaya. Last line — sort karke distinct laga diya, isse ek clean ascending unique array mil gaya, jo aage second-largest jaise problems me kaam aayega.

4. Practice Problems (Isi Array Par Try Karo)

Neeche wahi concepts practice karne ke liye problems di gayi hain jo diye gaye code me use hui hain. Pehle khud solve karne ki koshish karo, phir Section 5 me missing/important problems bhi dekho.

- 1. Array ko Stream API se ascending order me sort karke print karo.
- 2. Array ke saare elements ka sum, average, max aur min stream ke through nikalo.
- 3. Har element ka square nikalne wala ek naya array banao (map ka use karke).
- 4. Sirf even numbers ka ek unique (distinct) array banao.
- 5. Even numbers ke squares ka unique array banao aur unka total sum bhi nikalo.
- 6. Array me sabse bada even number nikalo.
- 7. Array me sabse chhota positive number nikalo.
- 8. Array ko sort karke uske duplicates hatao (ek clean ascending unique array banao).
- 9. Odd numbers ka sum stream ke through nikalo.
- 10. Count karo array me kitne even aur kitne odd numbers hain (do alag counts).

5. Important Problems Jo Code Me Missing The

Aapke code me last comment tha `// second largest Unique` — jo incomplete chhoda gaya tha. Ye aur kuch aise hi commonly-asked (interview/exam level) problems hain jo is topic se related hain lekin code me cover nahi hue the. Solution sabka diya gaya hai.

A) Second Largest Unique Element

```
int[] uniqueSorted = Arrays.stream(arr).sorted().distinct().toArray();
int secondLargest = uniqueSorted[uniqueSorted.length - 2];
System.out.println(secondLargest);
```

Pehle sort + distinct se ek unique ascending array banaya (jaisa code me pehle se tha), fir last se doosra element hi second largest hai.

B) Second Smallest Unique Element

```
int[] uniqueSorted = Arrays.stream(arr).sorted().distinct().toArray();
int secondSmallest = uniqueSorted[1];
System.out.println(secondSmallest);
```

Same unique sorted array me se index 1 second smallest dega.

C) Odd Numbers Ka Sum Aur Count

```
int oddSum = Arrays.stream(arr).filter(x -> x%2 != 0).sum();
long oddCount = Arrays.stream(arr).filter(x -> x%2 != 0).count();
System.out.println(oddSum + " " + oddCount);
```

D) IntSummaryStatistics — Ek Hi Line Me Sum, Avg, Max, Min, Count

```
IntSummaryStatistics stats = Arrays.stream(arr).summaryStatistics();
System.out.println(stats.getSum() + " " + stats.getAverage() + " "
    + stats.getMax() + " " + stats.getMin() + " " + stats.getCount());
```

Ye ek built-in class hai jo baar-baar alag-alag stream call karne ke bajaye sab kuch ek hi jagah de deti hai — aur `getAverage()` double return karta hai, isliye manual (sum/count) truncation ki zaroorat nahi padti.

E) Descending Order Me Sort Karna

```
Integer[] boxed = Arrays.stream(arr).boxed().toArray(Integer[]::new);
Arrays.sort(boxed, Collections.reverseOrder());
System.out.println(Arrays.toString(boxed));
```

`IntStream` sirf ascending sort deta hai, descending ke liye pehle `Integer[]` me box karna padta hai (`boxed()`), fir `Collections.reverseOrder()` use karte hain.

F) Frequency Count — Kaunsa Number Kitni Baar Aaya

```
Map<Integer, Long> freq = Arrays.stream(arr).boxed()
    .collect(Collectors.groupingBy(x -> x, Collectors.counting()));
System.out.println(freq);
```

`Collectors.groupingBy()` se har unique number ki frequency ek Map me mil jati hai — ye interview me bahut common pucha jata hai.

G) Even/Odd Ko Do Groups Me Partition Karna

```
Map<Boolean, List<Integer>> partitioned = Arrays.stream(arr).boxed()
    .collect(Collectors.partitioningBy(x -> x % 2 == 0));
System.out.println("Even: " + partitioned.get(true));
System.out.println("Odd: " + partitioned.get(false));
```

Collectors.partitioningBy() ek boolean condition ke basis par pura data do lists me todh deta hai — true wale aur false wale.

H) Array Ko Comma-Separated String Banana

```
String csv = Arrays.stream(arr).mapToObj(String::valueOf)
    .collect(Collectors.joining(", "));
System.out.println(csv);
```

mapToObj() IntStream ko Stream<String> me convert karta hai (kyunki joining() String Stream par kaam karta hai), fir Collectors.joining() sab ko comma se jod deta hai.

I) Kth Largest Unique Element (General Version)

```
int k = 3; // example
int[] uniqueSorted = Arrays.stream(arr).sorted().distinct().toArray();
int kthLargest = uniqueSorted[uniqueSorted.length - k];
System.out.println(kthLargest);
```

Second largest wale hi logic ko generalize kar diya — koi bhi k de kar us position ka largest element nikal sakte ho.

Tip: Practice karte waqt pehle plain loop se solve karo, fir stream version likho — isse dono approach clear ho jayengi aur interview me dono explain kar paoge.